Tech Trivia - Team Guide

Live quiz web app for the ACM auditorium event. Built with React, Express and MongoDB.

1. What this game is

Tech Trivia is a live quiz. Many students play at the same time on their own phones. Every student gets different random questions. The projector shows only a live leaderboard. The admin controls when the game starts, how long it runs and when it ends.

It replaces the old Flutter 'Coders Cup Tech Trivia' app with a version the whole team can read and change (React frontend, Node/Express backend, MongoDB database).

Main ideas:

- Students join with one common game PIN, their name and their roll number.
- Each student gets their own random set of questions, in their own order, with shuffled options.
- The whole game has one fixed end time. If you join late, you get less time.
- Every question has its own countdown. The server checks the time, not the phone.
- A faster correct answer gives more points.
- The correct answers never leave the server, so students cannot find them in the browser.

2. The pages

- Students (phone): the main link, for example `https://your-app.onrender.com/`. They join here and then play on `/play`.
- Projector: `/screen`. Shows the title, QR code, PIN, time left, how many joined and the top 10 leaderboard. Nobody needs to log in.
- Admin (laptop): `/admin`. Password protected. Import questions, change settings, Start, add time, close joining, End, Reset and download results.

3. How one game runs

Before the event:

1. Admin logs in and imports the questions CSV.
2. Admin sets the PIN, total time, seconds per question and number of questions per student, then saves.
3. Admin opens `/screen` on the projector.

Joining (lobby):

1. Students scan the QR code or open the link. They enter name, roll number and PIN.

2. The server gives each student their own random questions right away.
3. Students see a waiting screen. The projector shows how many have joined.

Playing:

1. Admin presses Start. The server sets the end time to now + total minutes.
2. Each phone shows one question at a time with a countdown bar and the game time left.
3. The student taps A, B, C or D, or the time runs out. The next question appears. The app never says if the answer was right or wrong.
4. Students can join after Start (until admin closes joining), but they share the same end time.
5. The leaderboard on the projector updates every few seconds.

Finishing:

1. A student finishes when all their questions are done, or when the game end time arrives.
2. The student sees score, correct answers, total time and current rank.
3. Admin can press End now at any time, or add 2 or 5 minutes while the game is running.
4. Admin downloads the results CSV. Reset clears all players but keeps the questions.

4. Rules

Scoring:

- Correct answer: 500 to 1000 points. Answering instantly gives about 1000, answering at the last second gives 500.
- Wrong answer or no answer: 0 points.
- Ranking: higher score first. If scores are equal, the lower total time wins.

Time:

- Each question has the same time limit (default 15 seconds, set by admin).
- The question timer starts when the server first sends that question to the student. Refreshing the page does not reset it.
- If time runs out, the question counts as wrong and the next one starts.
- There is a 1.5 second allowance for slow networks when an answer arrives.
- The game end time is the same for everyone. A late joiner can finish fewer questions. When the game ends, the current unanswered question is lost.

Other rules:

- One roll number = one player. Joining again with the same roll number is blocked, unless it is the same device returning after a refresh.
- Joining is blocked after the game has ended, or when admin closes joining.

5. Cheating protection

What we do:

- Every student has different questions in a different order, and the options are shuffled too, so looking at a friend's phone does not help.
- The correct answer stays on the server. The phone only gets the question text and options.
- The timer is checked on the server. Changing the phone's clock or refreshing does not help.
- Switching tab or leaving the browser shows a warning and is counted. The count appears in the results CSV (TabSwitches).
- Copy, cut, paste, right-click and text selection are blocked during questions.
- One attempt per roll number.

What we cannot fully stop:

- A student using a second phone to search Google or an AI tool. Short questions, a short timer and volunteers walking around the hall help the most.
- Tab switches are only recorded. They do not remove points automatically. The team decides what to do with them.

6. Tech stack and folders

Stack:

- Frontend: React 18, Vite, Tailwind CSS 4, React Router, KaTeX (math), PapaParse (CSV), qrcode.react (QR code).
- Backend: Node.js, Express 4, Mongoose (MongoDB), JSON Web Tokens (admin login).
- Database: MongoDB Atlas (free tier is enough for storage).
- Hosting: one Render web service. The Express server serves both the API (`/api/...`) and the built React files.

Folders:

- `server/index.js`: all API routes and server start.
- `server/logic.js`: scoring, timing and question shuffling (no database code, easy to test).
- `server/models.js`: MongoDB models (Question, Game, Player).
- `client/src/pages/Join.jsx`, `Play.jsx`, `Screen.jsx`, `Admin.jsx`: the four pages.
- `client/src/api.js`: helper for calling the API, token helpers and the mm:ss time formatter.
- `client/src/MathText.jsx`: shows maths written as `$...$` using KaTeX.
- `client/src/App.jsx`: routes and the background image with a dark overlay.
- `sample-questions.csv`: 64 ready questions.
- `README.md`: setup and deploy notes.

7. Frontend in detail

Join page (`/`):

- Three fields: name, roll number, PIN. Sends them to `POST /api/join`.
- On success the server returns a token. The token is saved in the browser (localStorage, key `tt_token`) and the student goes to `/play`.
- If a token already exists, the page goes straight to `/play`, so a refresh does not lose the player.

Play page (`/play`):

- It asks the server `GET /api/play` and shows what the server says. There are three phases: lobby, question and done.
- Lobby: waiting message. The page checks again every 3 seconds until Start.
- Question: shows the question number, game time left, a countdown bar (turns red in the last 4 seconds), the question text and four buttons.
- The countdown uses the time left given by the server. When it reaches zero the app sends an empty answer automatically.
- When the student taps an option, the app calls `POST /api/answer` with the question number and chosen option, then loads the next question.
- Done: score, correct count, total time and rank. It refreshes the rank every 8 seconds.
- If the server says the session is invalid (401), the token is removed and the student returns to the join page.
- Anti-cheating code runs only during questions: tab-switch detection (calls `POST /api/flag`) and blocking copy, paste and right-click.

Screen page (`/screen`):

- Calls `GET /api/leaderboard` every 3 seconds.
- Shows a big countdown, QR code and PIN (until the game ends), joined and finished counts, and the top 10 with medal colors for the top 3.
- Made for a projector: very large text.

Admin page (`/admin`):

- Login with the admin password. The token is kept in sessionStorage (`tt_admin`) and lasts 12 hours.
- Game Control: status, joined, finished, time left, and the buttons Start, +2 min, +5 min, Joining close/open, End now, Reset. The page refreshes every 3 seconds.
- Settings: PIN, total minutes, seconds per question, questions per student.
- Questions: upload a CSV (parsed in the browser with PapaParse and sent to the server), choose replace or add, preview, clear all.
- Results: download the CSV.

Math and code in questions:

- Text between `$...$` is shown as math (for example `$O(n^2)$`).
- New lines in a question are kept, so small code snippets work.

8. Backend in detail

Database collections:

- Question: text, four options, correct option number (0 to 3). This is the question pool.
- Game: one document only. Holds PIN, total minutes, seconds per question, questions per student, status (lobby or running), joinOpen, startedAt and endsAt. The game counts as 'ended' automatically when the current time passes endsAt.
- Player: name, roll number, secret token, their own list of questions (already shuffled, with the correct option number kept only on the server), current question number, when the current question started, score, correct count, total time, tab-switch count and a list of their answers.

Important design choice: each player's questions are copied into the player's own document when they join. So changing the question pool later does not break a running game, and the server needs no extra lookups while students play.

API for students:

- `POST /api/join`: checks PIN, roll number and game state, creates the player and returns a token.
- `GET /api/play`: returns lobby, the current question (with remaining time) or the final summary. It also marks expired questions as wrong.
- `POST /api/answer`: records the answer for the current question only. Duplicate or late requests are ignored safely. Returns only whether the player is done, never right or wrong.
- `POST /api/flag`: adds one to the player's tab-switch count.
- `GET /api/leaderboard`: public top 10, counts, PIN, status and end time. Cached for 2 seconds.

API for admin (all need the admin token except login):

- `POST /api/admin/login`: checks the password, returns a token.
- `GET /api/admin/game`: settings, status and counts.
- `POST /api/admin/settings`: PIN (up to 20 characters), total minutes (1 to 240), seconds per question (5 to 180), questions per student (1 to 100).
- `POST /api/admin/start`, `/extend`, `/end`, `/join-open`, `/reset`: game controls.
- `GET /api/admin/questions`, `POST /api/admin/questions/import`, `DELETE /api/admin/questions`: manage the pool.

- `GET /api/admin/results.csv`: download results (Rank, Name, Roll, Score, Correct, Answered, TotalTimeSec, TabSwitches, Finished).

How timing works on the server:

- When the server first sends a question, it saves the start time. The deadline is the earlier of start + seconds per question, and the game end time.
- On the next request, any question past its deadline is closed as wrong, and the player moves on.
- Updates to a player use a version check, so two requests at the same moment cannot corrupt the score.

Performance choices for a big crowd:

- Game settings and the question pool are cached in server memory for a short time.
- The leaderboard is cached for 2 seconds, so the projector and many viewers do not overload the database.
- Students only call the server when a question loads or is answered (and every 3 seconds in the lobby), so the load is spread out.

Security notes:

- Admin password and JWT secret live in environment variables, never in the code.
- Exported names beginning with =, +, - or @ get a leading apostrophe so Excel cannot run them as formulas.

9. Setup

You need: Node.js 20 or newer, a MongoDB Atlas account, a GitHub account and a Render account.

MongoDB Atlas:

1. Create a free M0 cluster.
2. Database Access: create a user with a simple password (letters and numbers).
3. Network Access: allow `0.0.0.0/0`.
4. Connect > Drivers: copy the connection string and add the database name `techtrivia` after `.net/`.

Environment variables (file `server/.env`, or Render settings):

- `MONGODB_URI`: the connection string.
- `ADMIN_PASSWORD`: the admin login password.
- `JWT_SECRET`: any long random text.
- `PORT`: 5000 (Render sets it by itself).

Run on your laptop:

1. `npm run install:all`
2. Terminal 1: `npm run dev:server`
3. Terminal 2: `npm run dev:client`, then open `http://localhost:5173`.

Deploy on Render (one web service):

1. Push the project to GitHub (never push `.env`).
2. New > Web Service > choose the repo.
3. Build command: `npm run install:all && npm run build`. Start command: `npm start`.
4. Add the environment variables and `NODE_VERSION=20`.
5. Use the Starter plan for the event. The free plan sleeps after 15 minutes without traffic and may be slow with a big crowd.

10. Testing checklist

1. Import the sample CSV. It should say 64 added.
2. Set a short test game: 3 minutes, 10 seconds per question, 5 questions.
3. Open `/screen` on one tab. Join with 3 or more players using different browsers, an incognito window and a phone. Use a different roll number for each.
4. Wrong PIN must be rejected.
5. The same roll number from another browser must be rejected.
6. Players wait in the lobby, and the joined count rises on `/screen`.
7. Press Start. All players should get questions. Different players should see different questions.
8. Do nothing for 10 seconds. The next question should come by itself.
9. Refresh in the middle of a question. The timer must not reset.
10. Switch to another tab and come back. A warning should appear.
11. Finish the quiz. The score and rank appear, and the leaderboard updates.
12. Join a new player after Start. They should get less total time.
13. Try +2 min, close joining, End now and Reset.
14. Download the results CSV and check the TabSwitches column.
15. After deploying, repeat the main flow on real phones using both WiFi and mobile data.
16. Do a small crowd test with 10 to 20 friends joining and starting at the same time.

11. Event day plan

- One day before: full test on the deployed site, final questions reviewed by the team.
- 1 hour before: open the site to wake the server, check `/admin` login, import the final questions, set PIN and settings.
- 15 minutes before: open `/screen` on the projector, test the QR code from a phone, check the hall WiFi.

- Announce the rules: one phone only, no tab switching, no help from others, mobile data is allowed.
- Start when most people have joined, then close joining when you decide.
- Keep 3 or 4 volunteers walking around the hall.
- After the quiz: End now if needed, download the results CSV, announce the top 3, then Reset.
- After the event: stop or delete the Render service so it does not keep costing money.

12. Known limits and risks

- The system has not been tested with a real MongoDB database or with hundreds of users yet. Test before the event.
- It is designed to run as a single server instance. Do not scale it to several instances without changes, because the short caches live in server memory.
- Roll numbers are not checked against a list. Anyone can type any roll number, and the first person to use a roll number gets it.
- The rank shown to a student when they finish is the rank at that moment, because others are still playing. The final leaderboard is on the projector.
- The number of questions per student is fixed when they join. Set the questions and settings before students join. The seconds per question is read live.
- Hall WiFi can be the weakest part. Tell students they can use mobile data.
- The sample questions were written for this project and should be reviewed by the team before the event.
- The admin password protects the admin page. Share it only with the team.

13. Suggested team split

- Backend and deploy: set up Atlas and Render, environment variables, watch the server during the event.
- Frontend: check the student, projector and admin screens on different phones and screen sizes, and polish colors and text.
- Questions: review the 64 questions, replace weak ones, add more, and keep a good mix of easy, medium and hard.
- Event operations: projector and sound, WiFi, volunteers, announcing rules and winners.

Everyone should do at least one full test run as a student and once as admin.

14. Differences from the old Flutter app

- Old app: students entered only a code, played at their own speed and got 10 random questions with one stopwatch. New app: name, roll number and a common PIN, a shared game clock, and a timer for every question.
- Old app: the score was 10 per correct answer plus a small time bonus, calculated on the phone. New app: points per question based on speed, calculated on the server.
- Old app: the correct answers were sent to the phone. New app: they stay on the server.

- Old app: Firebase with Firestore. New app: Express with MongoDB.
- Same in both: CSV import with the format question, four options and correct option number (0 to 3), maths support, a dark theme and the Tech Trivia background image.